Bounded Vertical Agents

A State-and-Control Architecture for Safer Real-World AI Agents

Yue Li

Founder and AI Agent Developer, U Agent | San Jose, California | yueli555github@icloud.com

Working Paper / Research Note

Abstract

Current agent systems are often evaluated through the apparent quality of model output, yet many real-world failures emerge from the software architecture surrounding the model rather than from language generation alone. This paper argues that vertical AI agents should be treated as bounded execution systems, not general-purpose autonomous workers. A practical agent in a customer-facing or business-critical domain must operate inside a narrow business boundary, separate model reasoning from execution authority, maintain durable task state, and route irreversible actions through a single controlled path with human confirmation. Drawing from the design and implementation of U Agent, a production AI email workflow system, this paper proposes a state-and-control architecture for vertical agents: task truth, stage detection, policy gates, action events, runtime projection, evaluation harnesses, and explicit final action gates. The central claim is that harness engineering does not eliminate hallucination or model uncertainty; it reduces blast radius by making failures bounded, observable, reversible where possible, and recoverable. The paper outlines principles, failure modes, architectural requirements, and an empirical research agenda for testing safer vertical agent runtimes.

Keywords: AI agents; vertical agents; human-in-the-loop systems; agent safety; state machines; tool use; workflow automation; bounded autonomy; evaluation harnesses.

1. Introduction

AI agents are often described as systems that can plan, call tools, and complete tasks with minimal human intervention. In practice, however, the most difficult problems are rarely the visible parts of the agent, such as producing fluent text or selecting a tool. The deeper challenge is architectural: how can a probabilistic model be safely connected to deterministic business workflows, external systems, and irreversible actions?

Traditional software systems usually assume that state is explicit, deterministic, and controlled by code. Agent systems introduce a different kind of uncertainty. Their behavior depends on model outputs, prompts, context windows, tool responses, memory, task decomposition, and prior intermediate states. This makes real-world agent behavior difficult to predict, reproduce, and verify. The result is a mismatch between traditional software architecture and agentic execution.

This paper argues that professional vertical agents should not be designed as unconstrained autonomous workers. They should be designed as bounded execution systems. In this model, the LLM may reason, summarize, draft, classify, and recommend, but it should not own final authority over high-impact actions. The surrounding product architecture must provide task truth, stage detection, policy gates, event logs, user review, and recovery paths.

The claims in this paper come from practical experience building U Agent, an AI email assistant designed around human-approved execution for real sales workflows. U Agent is not used here as a benchmark result

or a formal controlled experiment. Rather, it serves as an implementation case study that motivates a broader architectural thesis: vertical agent safety depends as much on state-and-control design as on model capability.

2. The Core Thesis

The core thesis is simple:

> *Vertical AI agents should not be treated as general-purpose autonomous workers. They should be designed as bounded execution systems, where model reasoning is separated from state truth, policy gates, action authority, and human-confirmed irreversible actions.*

This thesis has two implications. First, raw AI output is not system truth. A model saying that a draft is ready, a customer replied, or an action should be taken does not make those facts true. Second, a tool call is not the same as safe execution. Tool access must be mediated by product-level policy, identity, permissions, workflow state, and human confirmation when actions are consequential.

3. Vertical Agent Principles

Narrow business boundary

An agent should operate only inside the product-authorized business domain. It should not freely inspect unrelated systems, unrelated inbox messages, or external resources outside the user-approved workflow.

Single execution path for real-world actions

Irreversible or customer-facing actions, such as sending email, updating customer status, or producing official replies, should pass through one controlled execution path. There should be no alternate route through raw model text, UI shortcuts, or unverified tool output.

Runtime state over raw AI output

A vertical agent needs durable task truth, stage detection, policy gates, action events, and auditable state transitions. The runtime state, not the language model alone, should determine whether an action is available.

Harness engineering reduces blast radius

Evaluation and runtime harnesses do not make model behavior perfectly reliable. Their purpose is to bound failure, create traceability, support safe retries, and enable recovery after interrupted or incorrect runs.

RAG is access, not intelligence

Retrieval can provide knowledge, but domain performance depends on judgment, workflow control, business state, and execution policy. Adding more documents does not automatically create a reliable agent.

Domain-specific action models are required

Architectures built for coding agents cannot be blindly copied into sales, legal, medical, finance, or other customer-facing domains. Each vertical domain has a different risk model, action model, permission boundary, and acceptable failure mode.

Unrestricted autonomy is not yet appropriate for high-impact individual work

General-purpose autonomous agents remain too difficult to constrain, observe, and verify for many professional workflows. Safer systems should be narrow, observable, and human-confirmed for high-impact actions.

Agent learning must be controlled

Agents may learn user preferences and improve recommendations, but they must not self-expand permissions, rewrite safety rules, or silently change execution policy.

4. Why Traditional Software Struggles With Agents

Traditional software is usually built around deterministic state transitions: a button is clicked, a request is sent, a database row is updated, and a UI reflects the result. Agent behavior is different. A model may infer intent, construct intermediate plans, revise its reasoning, call tools, summarize context, or produce partially correct outputs. The execution state is therefore partly generated by a probabilistic system.

This creates a structural mismatch. If traditional software treats model output as if it were deterministic state, the system becomes fragile. The model may misread the user request, confuse two tasks, operate with stale context, call a tool at the wrong time, or produce a plausible but unsupported claim. In a pure chat interface, this may be annoying. In an automated workflow, it can become destructive.

For agents to work reliably, traditional software must evolve. It needs state machines designed for agent behavior: stages that represent uncertain progress, policy checks that prevent unsafe transitions, event logs that preserve facts, projections that make runtime state visible, and final gates that decide whether an action is allowed. This is not a cosmetic change. It changes what the application considers authoritative.

5. Failure Modes in Real-World Agent Workflows

The most important agent failures are often not failures of grammar or fluency. They are failures of state, authority, identity, permission, or recovery. The following failure modes are especially important in vertical domains.

- **Task-state drift:** The model believes a task is complete, blocked, or ready when the durable workflow state says otherwise. Control: durable task truth and stage validation.
- **Permission overreach:** The agent attempts to inspect or act outside the authorized workflow boundary. Control: provider scope, workspace identity, and policy gates.
- **Unsafe tool timing:** The model calls a tool before required prerequisites are satisfied. Control: state-machine transitions and precondition checks.
- **Irreversible action error:** The agent sends, submits, deletes, purchases, or updates something incorrectly. Control: single execution path and human confirmation.
- **Stale context:** The agent acts on old information after a newer reply, cancellation, or state change. Control: event freshness checks and replay-safe hydration.
- **Multi-agent confusion:** Multiple agents create conflicting responsibilities, duplicate work, or misleading intermediate claims. Control: clear ownership model and verifiable final authority.

6. A State-and-Control Architecture for Bounded Agents

A bounded vertical agent runtime should separate reasoning, state, display, and execution authority. The following architecture is one way to make that separation explicit.

- **1. Reasoning layer:** The LLM interprets the user request, reads authorized context, drafts responses, and recommends next actions. This layer is useful but not authoritative for irreversible execution.
- **2. Task truth layer:** A structured representation of what the task is, what stage it is in, which identity it belongs to, and whether it is ready, blocked, failed, or completed.
- **3. Action event ledger:** An append-only record of durable facts: messages sent, replies received, drafts prepared, failures, dismissals, confirmations, and other state-changing events.
- **4. Policy and permission gates:** Checks that enforce workspace identity, provider identity, allowed data access, freshness, user consent, and domain-specific restrictions.
- **5. Runtime projection:** A display layer that explains what the agent is doing, what happened, and what action is available. It should not be the source of send or execution authority.
- **6. Final execution gate:** The only path through which an irreversible action occurs. For high-impact actions, this path should require human review and explicit confirmation.
- **7. Evaluation harness:** Tests, fixtures, smoke checks, and replay scenarios that evaluate state transitions, recovery behavior, and failure containment across repeated agent runs.

7. Implementation Case Study: U Agent

U Agent is a production AI email assistant built around human-approved execution. Its practical design problem is simple but difficult: an AI system may help analyze sales conversations and prepare follow-up drafts, but it must not become a general inbox reader and must never send emails without user approval.

This constraint leads to a bounded architecture. The agent can reason over authorized conversation context, detect relevant replies, prepare a draft, and surface the draft to the user. However, the final send action is not controlled by raw assistant text. A sendable draft must satisfy structured task state, identity, recipient, subject, body, message reference, and freshness requirements before the backend send gate allows execution.

This design turns an AI email assistant into a testbed for a broader question: how should vertical agents operate when real-world actions require deterministic accountability but model reasoning remains probabilistic? The answer is not merely better prompting. It is a runtime design that treats the model as one component inside a controlled execution system.

8. Multi-Agent Systems: Why More Agents Can Mean Less Reliability

Multi-agent systems are attractive in theory because they promise role specialization: one agent plans, another researches, another critiques, and another executes. In practice, however, multi-agent systems can increase unreliability if responsibility boundaries are unclear. Agents may duplicate work, pass along unsupported assumptions, drift away from the original goal, or produce a final output that is hard to verify.

The central issue is not the number of agents. It is authority. A system with many agents still needs a single source of task truth, a clear ownership model, and a final execution gate. Without those controls, adding more agents can increase communication overhead and reduce accountability. In high-impact workflows, multi-agent collaboration should be evaluated not only by output quality but also by traceability, reproducibility, and failure containment.

9. Evaluation Agenda

The principles above should be evaluated empirically. A useful research program would compare different vertical-agent runtime architectures across the same set of tasks and measure not only success rate, but also failure containment and recovery behavior.

Possible evaluation questions include:

- Does separating raw model output from task truth reduce unsafe action availability?
- How often do agents enter task-state drift under interrupted, stale, or ambiguous contexts?
- Do explicit policy gates prevent permission overreach without blocking legitimate workflow progress?
- How does a single final execution gate affect reliability in email, customer-support, or document-processing workflows?
- What types of harness tests best predict production failures in vertical agent systems?
- When do multi-agent architectures improve task performance, and when do they increase unverifiable complexity?

Metrics should include successful task completion, unsafe action attempts blocked, stale-context errors, recovery success after interruption, duplicate action prevention, human override rate, and auditability of final decisions. The goal is not to prove that agents never fail. The goal is to measure whether failures remain bounded, visible, and recoverable.

10. Implications

If vertical agents require state-and-control architectures, then the next stage of agent engineering will not be solved by model capability alone. Stronger models may reduce some errors, but they do not remove the need for product-level control. In fact, more capable agents may require stronger gates because they can affect more systems and take more convincing but incorrect actions.

This has implications for companies adopting agents in sales, legal, medical, finance, customer support, and software engineering. A safe deployment should start with narrow scope, explicit permissions, observable runtime state, and human confirmation for high-impact actions. It should avoid unrestricted autonomy until the system has demonstrated reliable behavior under realistic failure cases.

11. Limitations

This paper is a working research note, not a completed controlled study. The claims are derived from practical implementation experience and architectural analysis rather than a formal benchmark. U Agent is used as a case study and motivating system, not as proof that one architecture is universally optimal. Future work should validate these ideas with controlled experiments, multiple domains, independent implementations, and quantitative evaluation.

Another limitation is that different vertical domains have different risk models. Email drafting, healthcare triage, financial decision-making, and autonomous coding all require different action gates and evaluation standards. The principles in this paper are intended as a general architecture pattern, not a replacement for domain-specific safety analysis.

12. Conclusion

Agents are not useless, but they should not be used blindly. They are most promising in narrow, well-scoped, observable vertical domains where the system can define what the agent may access, what it may recommend, and what it may execute.

The central lesson is that agent safety is not only a model problem. It is also a software architecture problem. Vertical agents need durable task truth, controlled execution paths, policy gates, event logs, runtime projections, and human confirmation for irreversible actions. Harness engineering cannot eliminate uncertainty, but it can reduce blast radius and make failures traceable.

When probabilistic agents are connected to deterministic business workflows, both sides must change. Software must become more agent-aware, and agents must become more bounded, observable, and accountable. That is the direction in which practical agent systems should evolve.

Author note

This working paper summarizes research ideas developed through building U Agent, a production AI email workflow system focused on human-approved execution, bounded mailbox access, state-machine-controlled runtime behavior, and safe handling of customer-facing drafts. It is intended as a preliminary research note for discussion and further empirical development.